

Implementation and Evaluation of PRM and RRT Path Planning in Static and Dynamic Maze Environments

Kim Svedberg*, Zebastian Thorsén†
School of Innovation, Design and, Engineering
Mälardalens University, Västerås, Sweden
Email: *Ksg22001, †Ztn22002@student.mdu.se

Abstract—This paper presents the implementation and evaluation of two sampling based path planning algorithms: Rapidly exploring Random Trees (RRT) and Probabilistic Roadmap Method (PRM), applied in two custom mazes. Both a static and dynamic version of each maze are evaluated, where in the dynamic version the map changes in the middle of the simulation to force a replanning event. The performance of both algorithms is measured across three metrics: path length, planning time, and total execution time. Both planners successfully reached the goal in all scenarios. The results show that PRM consistently produced shorter paths and lower total execution times across all configurations, while RRT achieved faster planning times due to its lower upfront construction cost.

Index Terms—Path planning, Robotics, RRT, PRM, Sampling-based path planning

I. INTRODUCTION

In robotics, path planning is a fundamental part of behaviour control. It is important for navigation of the robot and for autonomous implementation. To navigate without collision from a starting position to a goal position, a path which the robot should take is decided. The problem that arises is finding a path which is collision-free, while finishing in reasonable time.

Classical grid-based algorithms such as Dijkstra and A* discretize the configuration space into a graph and search exhaustively [1]. While this will produce a path to the goal, and in the case of Dijkstra and A* will give an optimal path, these approaches scale poorly with high-dimensional maps and require rigid grid representation of the environment.

Sampling-based planners address these limitations by avoiding explicit discretization of the entire configuration space [2]. Instead of building a full map, they randomly sample configurations and connect them via local planners subject to collision checks. This makes them very effective in high dimensional and continuous spaces where grid-based methods becomes too computationally expensive to run, and provides probabilistic completeness: as the number of samples increases, the probability of finding a solution if one exists approaches one. Some of the most influential sampling-based planners to date are the Probabilistic Roadmap Method and the Rapidly-exploring Random Trees [3].

Probabilistic Roadmap Method (PRM), introduced by

Kavraki et al. in 1996, is a multi-query planner that operates in two phases [4]. In the construction phase, it builds a roadmap of the free configuration space by randomly sampling nodes and connecting nearby pairs whose straight-line path is collision free. In the query phase, a graph search algorithm, like Dijkstra or A*, finds the shortest path through this roadmap from start to goal. Because the roadmap is reusable, PRM is particularly suited for environments where multiple queries must be answered or where replanning may be required during execution.

Rapidly-exploring Random Trees (RRT), introduced by LaValle in 1998, is a single-query planner that grows a tree incrementally from the start position [5]. At each iteration, a random sample is drawn from the free space, and the tree is extended toward it by a fixed step size if there are no collision. This causes the tree to rapidly explore the space, making RRT effective in large, complex environments without any upfront construction cost.

Comparing PRM and RRT directly, Jermyn conducted a controlled evaluation of both planners across 450 trials in a two dimensional (40×40) static environment, measuring path length, runtime, node count, and number of iterations [6]. The study found that RRT produced significantly longer paths (mean 39.05 units) than PRM (30.69 units), as RRT terminates immediately after reaching the goal without optimizing the path. PRM, using Dijkstra, consistently found more direct paths. However, RRT was much faster at runtime (mean 0.007 s vs 0.482 s), as it avoids the upfront cost of roadmap construction.

This paper implements both PRM and RRT in MATLAB and evaluates them on two custom maze environments under both static and dynamic conditions [7]. In the dynamic case, obstacles are introduced during movement to simulate newly detected obstacles, requiring the planner to replan the path in the middle of execution. Performance of both algorithms is assessed across three metrics: final path length, plan time and total execution time.

II. METHOD

Two path planning algorithms, PRM and RRT, were implemented and evaluated in MATLAB for both static and dynamic environments.

A. Map Representation and Obstacle Inflation

Two mazes with a static and dynamic variant were created for the project. Both are 80×80 binary occupancy grids in which 1 denotes an obstacle and 0 denotes free space, with different known starting and goal positions.

Maze 1 is a network of passage obstacles, which can be seen in Figure 1.

Maze 2 features a more open layout, with scattered islands of obstacles, which can be seen in Figure 2.

The dynamic variants of the mazes use the static variant but obstacles will appear, forcing a replan for both planners.

To account for the physical size of the robot, obstacle inflation is applied to the map prior to path planning. By expanding each obstacle uniformly by a safety clearance of one cell in all directions, the environment is converted into a configuration space where the robot can be safely treated as a point. This guarantees that the final generated path maintains a safe distance from all obstacles during navigation.

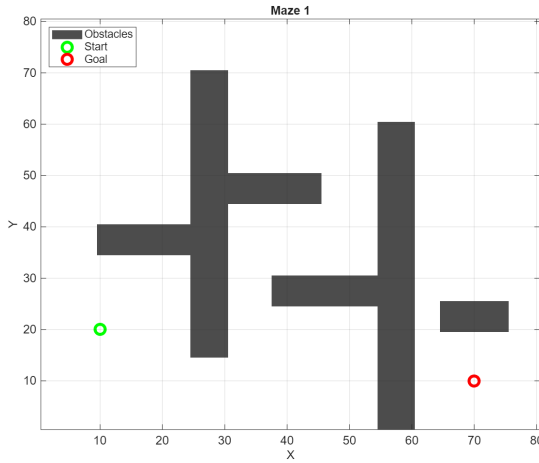


Figure 1. Maze 1. The green dot is the starting position of the robot and the red dot is the goal.

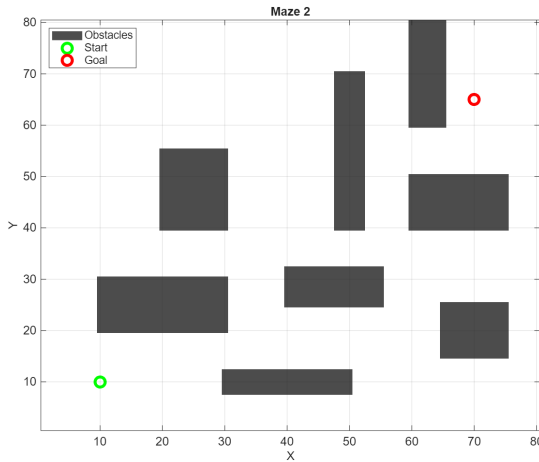


Figure 2. Maze 2. The green dot is the starting position of the robot and the red dot is the goal.

B. Probabilistic Roadmap

The first implemented planner was the Probabilistic Roadmap (PRM), which is a sampling-based path planning algorithm [4]. It works by using two separate phases, one construction/learning phase and a query phase. In the construction phase, it constructs a graph consisting of the nodes and edges, with the graph denoted as $G = (V, E)$, where V is the set of valid nodes and E represents the valid edges. To construct this graph, it does both node sampling and edge connections.

- 1) **Node Sampling (V):** It randomly samples points ($N = 250$ nodes) as coordinates (x, y) inside the boundaries of the 80×80 maze to build a mathematical representation of it. It performs a map inflation, inserting a safety distance around the walls/obstacles of the maze, with clearance set to 1. It discards points which are put on the inflated obstacles ($M_{\text{inflated}} = 1$), and only adds the points which are in open space ($M_{\text{inflated}} = 0$). The sampling of nodes continues until $N = 250$ valid nodes are distributed across the map.
- 2) **Edge Connection (E):** In the next step, it looks at all neighbouring nodes which are within a circular boundary with a maximum radius of $r = 22$ units, and connects the nodes by creating a line between them, only allowing lines which do not intersect obstacles. An $N \times N$ adjacency matrix (`adjMatrix`) is initialized with infinity values of all edges, and is used to constantly track the edge costs. By using the norm of the Euclidean distance, the distance between each node combination (i, j) is calculated as:

$$d(i, j) = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2} \quad (1)$$

If the distance is then within the circular boundary and the edge is clear of obstacles, then the distance is saved to the adjacency matrix.

To compute the final path, a graph search is performed on the computed adjacency matrix. It performs Dijkstra's search algorithm using MATLAB's optimized `shortestpath` function which extracts the shortest sequence of nodes within the graph from node 1 (Start) to node 2 (Goal).

C. Rapidly Exploring Random Tree

The second planner implemented was the Rapidly Exploring Random Tree (RRT) [5]. RRT works by growing a tree incrementally from the starting position, generating random samples in the free space and extending toward them. At each iteration, a random configuration q_{Rand} is drawn uniformly from the map. The nearest existing tree node q_{Near} is identified by Euclidean distance, and a candidate node q_{New} is extended from q_{Near} towards q_{Rand} by a fixed step size of 2 (or directly on q_{Rand} if closer). If the path from q_{Near} to q_{New} is collision free, meaning no obstacle exists between them, q_{New} is added to the tree with q_{Near} as its parent pointer.

Termination occurs when a new node falls within the goal threshold (2 units), at which point the goal is appended and

the final solution path is recovered by tracing parent pointers from the goal to the start. The maximum of iterations for RRT is set to 10,000 for each tree.

D. Dynamic Environment and Replanning

The dynamic environment is implemented by scripting two obstacle spawning events tied to the robot's progress in the maze. The first obstacle spawns at 20% of path completion in a region that should not intersect an optimized path (upper left corner of the maze), simulating a newly detected obstacle far away from the robot. The second obstacle is placed directly in front of the robot path at 55% completion, obstructing the pre-planned path and forcing a replanning event to complete the path to the goal.

After every step the robot takes, each consecutive waypoint along the remaining path is subject to a collision check by the planner, checking for collisions against the updated map. The two path planners handle a blocked path differently.

For RRT, if any segment is found to be blocked, the planner immediately discards the current path and tree, and grows a new tree from the robot's current position to the goal using the updated map.

For PRM, rather than rebuilding the entire roadmap from scratch, an online edge-discarding strategy [8] is implemented in the robot's execution loop. The blocked edge is removed directly from the existing graph, and the shortest-path query is re-run on the updated graph using Dijkstra's search algorithm. The process is then repeated until a path without collisions is found. Since the roadmap nodes and all remaining valid edges are preserved, this approach avoids the cost of resampling and reconnecting the full configuration space.

E. Performance Evaluation

To account for the stochastic nature of both planners, each planner was executed five times per maze for both the static and dynamic environment, and mean values were reported.

Both planners are evaluated across three metrics: path length, plan time and total execution time. Path length is the total distance travelled by the robot along the final path, computed as the sum of Euclidean distances between nodes. Plan time is the time it takes from the start of planning to the moment a valid path to the goal is found. Total execution time is measured from the start of the simulation to the moment the robot reaches the goal, including the plan time and robot traversal of the path.

III. RESULTS

To compare the different path planning algorithms in both the static and dynamic mazes, three metrics are evaluated: planning time, total execution time, for the specific planner, and path length of the current maze. As there are two different mazes, both consisting of a static and dynamic version, four cases can be evaluated for each planner.

A. Static Maze Results

The average resulting paths through static mazes 1 and 2 for both planners can be seen in Figures 3, 4, 5 and 6. The PRM

consistently found a shorter path to the goal compared to the RRT in both mazes, with about 20.3% and 36.4% shorter paths in Maze 1 and Maze 2 respectively. The mean statistics for both planners on both static mazes can be seen in Table I.

Planner	Maze	Path Length	Plan Time (s)	Total Time (s)
PRM	1	155.23	0.030	1.55
RRT	1	194.76	0.018	11.65
PRM	2	95.32	0.030	1.24
RRT	2	149.77	0.012	8.68

Table I
STATIC MAZES RESULTS FOR PRM AND RRT.

B. Dynamic Maze Results

The initial paths for both planners in the dynamic environments Figures 7, 8, 9 and 10 are similar to the static results, but have a forced replanning event in the middle of the travel due to obstacle spawning. The PRM in the dynamic environment also found the shorter path to the goal compared to the RRT in both mazes, regardless of the replanning. The results show about 24.1% and 31.9% shorter paths in Maze 1 and Maze 2 respectively. The mean performance statistics for both planners on both dynamic mazes can be seen in Table II.

Planner	Maze	Path Length	Plan Time (s)	Total Time (s)
PRM	1	168.87	0.031	2.00
RRT	1	222.46	0.020	13.06
PRM	2	113.66	0.037	1.61
RRT	2	166.82	0.014	9.89

Table II
DYNAMIC MAZES RESULTS FOR PRM AND RRT.

IV. DISCUSSION

The results across all four maze configurations consistently show that PRM produced shorter paths than RRT, both in the static and dynamic variants.

In a static environment, PRM achieved path length of 155.23 and 95.32 units in Maze 1 and Maze 2 respectively, compared to 194.76 and 149.77 units for RRT. This gap increased in the dynamic environment, where RRT's path grew to 222.46 and 166.82 units, while PRM, in comparison, remained more stable at 168.87 and 113.66 units. This is consistent with the findings by Jermyn, who found that RRT produced a significantly longer path (mean 39.05 units) than PRM (30.69 units) across 450 trials [6]. This is because RRT terminates immediately upon reaching the goal without optimizing the path, producing a longer and more winding path through the environments. PRM, in contrast, runs Dijkstra over its roadmap, consistently finding more direct routes toward the goal through the free space.

In terms of planning time, RRT was faster than PRM in all cases, achieving 0.012-0.020 s compared to 0.030-0.037 s for PRM. These results are expected, as RRT begins expanding immediately from the start with no upfront cost, while PRM must first sample $N = 250$ nodes, check all connections within radius $r = 22$, and build the full roadmap before any path query can be made.

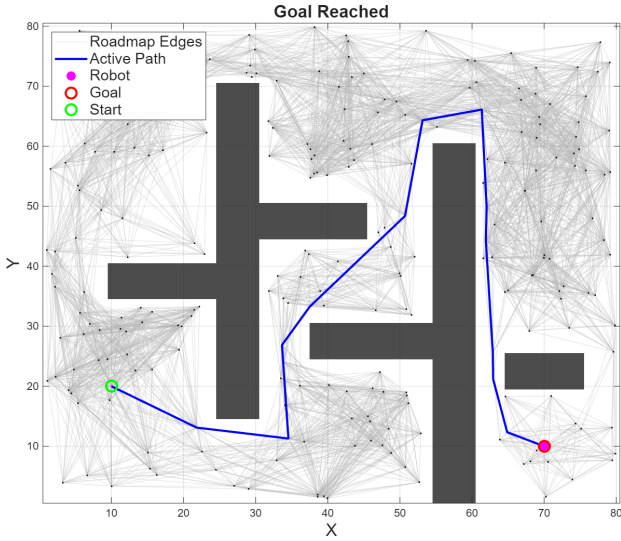


Figure 3. PRM path through static maze 1.

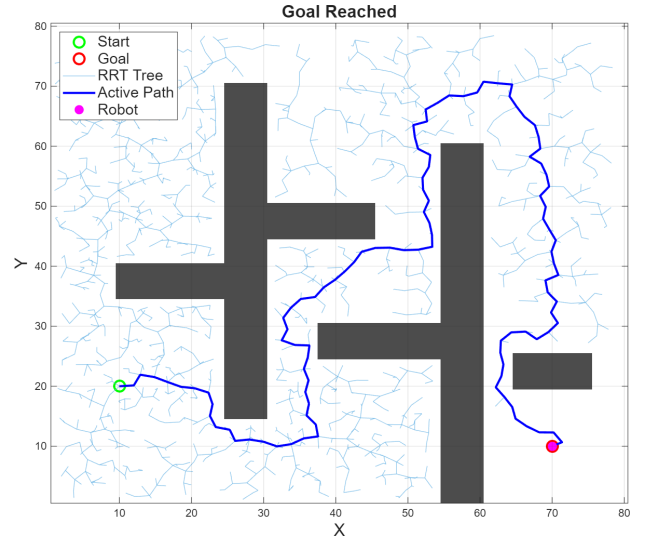


Figure 4. RRT path through static maze 1.

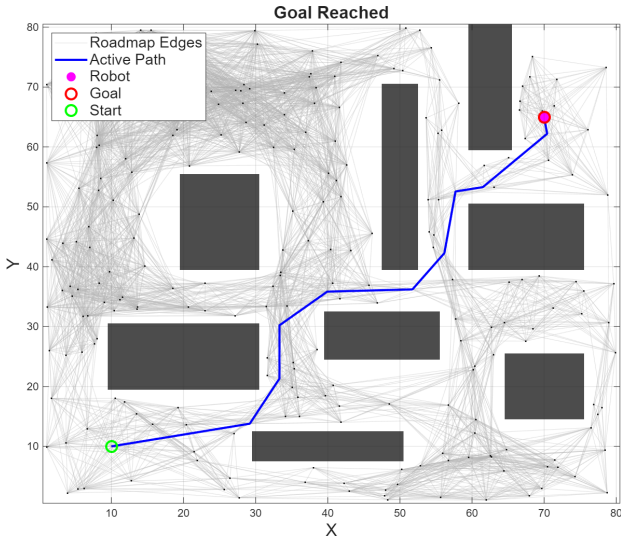


Figure 5. PRM path through static maze 2.

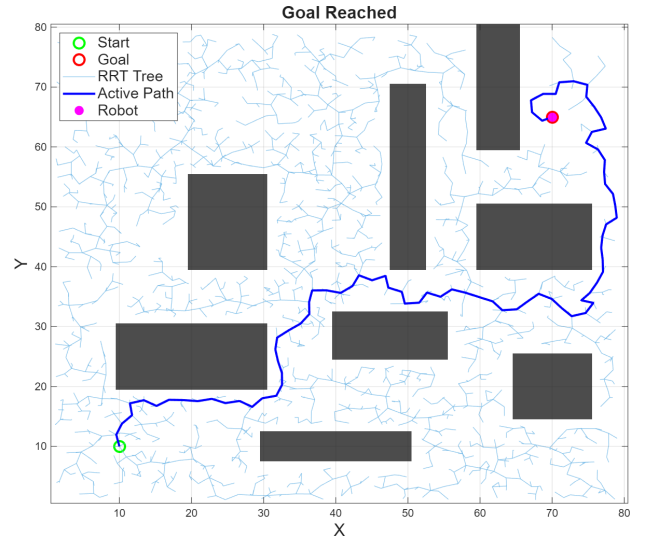


Figure 6. RRT path through static maze 2.

However, it is clear that the PRM performed the best in terms of execution time, as it was the shortest among all maze configurations, both static and dynamic.

Part of the reason that PRM was consistently faster was that its implementation of the path was not based on a predetermined step size, and rather on the distance between the nodes. When it traversed between two distant nodes, the single step was of a large step size, resulting in higher speed. It navigated slower in tight clusters of nodes as it had to visit and stay at each one for a fixed period of time. Apart from this consideration, it would still provide the same conclusion, that PRM performs the path slightly faster, mainly thanks to the way the dynamic replanning is implemented with edge pruning.

As the PRM does not have to reconnect the entire configuration space, it has a substantial time and processing advantage

in comparison to the RRT. The RRT deletes the entire tree, and rebuilds it from the current node. PRM on the other hand, only prunes edges which are blocked by the new obstacle, keeping the initially computed edges between all other nodes. If the PRM would have been implemented as standard and not as edge pruning PRM in the dynamic maze, then the total execution time would have increased as it would remove all connections when faced with an obstacle, similar to the RRT.

Another consideration is that the success of the PRM depends entirely on the node distribution. It can therefore struggle to find paths in environments where there are narrow passages if nodes are distributed too far from those passages. If the nodes are too distant, or the maximum radius is too small, then the algorithm might not be able to generate a final path to the goal. This underlines the importance of good parameter tuning, and the difficulty of guaranteeing success when relying

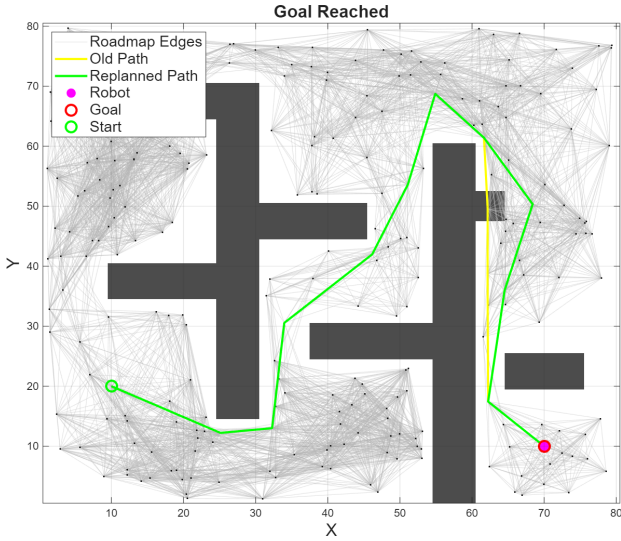


Figure 7. PRM path through dynamic maze 1.

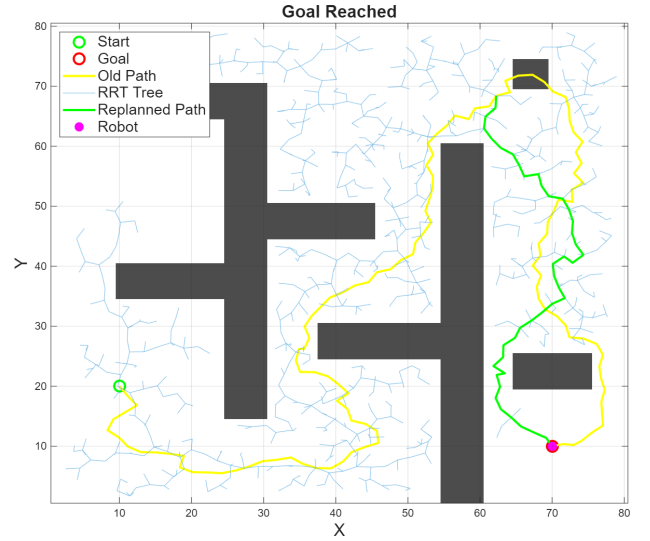


Figure 8. RRT path through dynamic maze 1.

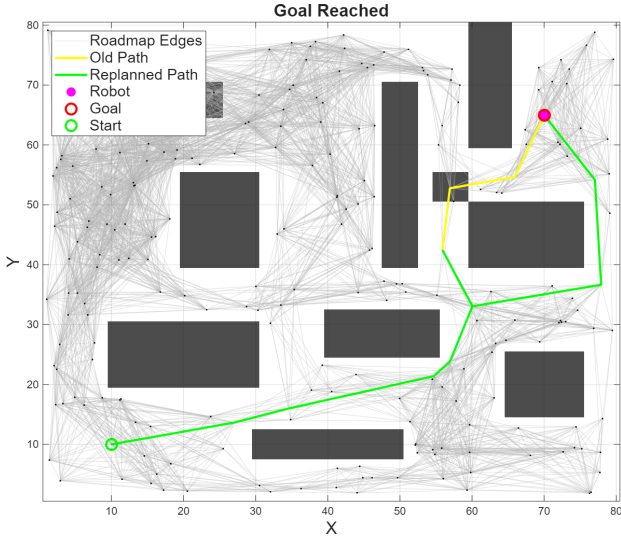


Figure 9. PRM path through dynamic maze 2.

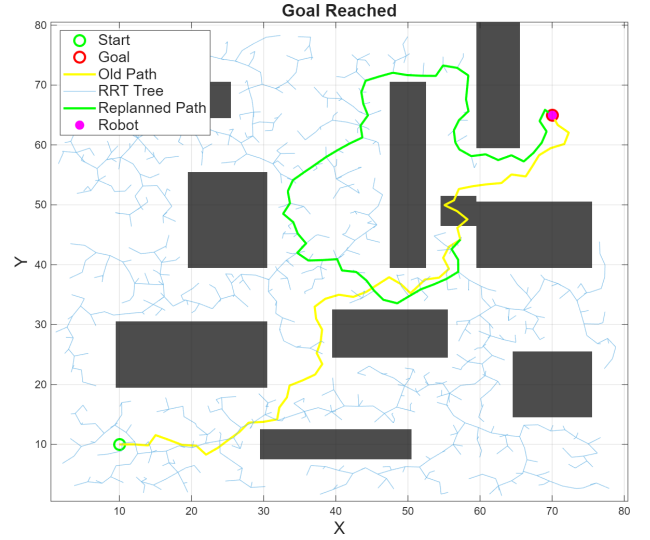


Figure 10. RRT path through dynamic maze 2.

on probability.

A. Future Work

Several extensions could improve both planners. RRT has a well known variant, RRT^* , which adds a rewiring step after adding a node to the tree. When a new node is added, RRT^* checks whether nearby nodes can be reached at lower cost through the new node, and updates their parent pointer accordingly. As the number of samples grow, RRT^* converges asymptotically to the globally optimal solution. A simpler improvement is to introduce goal bias, where there is a small fixed probability of sampling the goal instead of at random. This could reduce the number of iterations needed to reach the goal, particularly in open environments.

Similarly, PRM has a variant called PRM^* , which scales the connection radius as a function of node density to guar-

tee asymptotic optimal roadmap query. This could improve the quality of the queried path. Finally, evaluating both planners on more mazes across multiple trials would provide more robust conclusions.

V. CONCLUSION

In conclusion, this paper implemented and evaluated PRM and RRT across two mazes under static and dynamic conditions. Both planners reached the goal without collision, showing that sampling-based path planning can be used for dynamic path planning.

PRM consistently found shorter paths than RRT across all scenarios, with mean path lengths of 155.23 and 95.32 units in the static mazes compared to 194.76 and 149.77 for RRT. This is due to PRM's Dijkstra query over its roadmap, which minimizes cumulative path cost, while RRT terminates

immediately upon reaching the goal without path optimization. PRM also achieved lower total execution times in all cases, largely due to fewer waypoints to traverse and its edge-pruning replanning strategy in dynamic scenarios. RRT was faster at planning time, confirming that its single-query expansion has a lower upfront cost than PRM's roadmap construction.

REFERENCES

- [1] I. Munasinghe, A. Rodrigo, and A. Perera, "Experimental evaluation of grid-based path planning algorithms for mobile robot navigation," 09 2025.
- [2] A. Alexander, K. Venkatesan, J. Mounsef, and K. Ramanujam, "A comprehensive survey of path planning algorithms for autonomous systems and mobile robots: Traditional and modern approaches," *IEEE Access*, vol. PP, pp. 1–1, 01 2025.
- [3] E. F. Sertac Karaman, "Sampling-based algorithms for optimal motion planning," 05 2011.
- [4] L. Kavraki, P. Svestka, J. Latombe, and M. Overmars, "Probabilistic roadmaps for path planning in high-dimensional configuration spaces," *Robotics and Automation, IEEE Transactions on*, vol. 12, pp. 566 – 580, 09 1996.
- [5] S. M. LaValle, "Rapidly-exploring random trees : a new tool for path planning," *The annual research report*, 1998. [Online]. Available: <https://api.semanticscholar.org/CorpusID:14744621>
- [6] J. Jermyn, "A comparison of the effectiveness of the rrt, prm, and novel hybrid rrt-prm path planners," *International Journal for Research in Applied Science and Engineering Technology*, vol. 9, pp. 600–611, 12 2021.
- [7] "MATLAB home page," accessed: 23-05-2026. [Online]. Available: <https://www.mathworks.com/products/matlab.html>
- [8] R. Bohlin and L. E. Kavraki, "Path planning in the presence of obstacles: A lazy prm approach," in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, vol. 1, 04 2000, pp. 281–286.